

Multi-head attention

↓

Instead of having a single triplet (w_q, w_k, w_v)

and

~~$(w, d \times d_v)$~~

Multiple triplets (or attention blocks)

$(w_{q_1}, w_{k_1}, w_{v_1}), (w_{q_2}, w_{k_2}, w_{v_2}), \dots, (w_{q_m}, w_{k_m}, w_{v_m})$

↓
 $(x_1^v)_{1 \times d_v} \quad (x_2^v)_{1 \times d_v} \quad (x_m^v)_{1 \times d_v}$

$$\text{head}^i = (\underline{x_1^v} \odot x_2^v \odot x_3^v \dots \odot \underline{x_m^v})$$

$$y^i_{1 \times d} = (\text{head}^i)_{1 \times m d_v} \underline{W}_{m d_v \times d}$$

final output of Multi-head attention block

dimension of the model

Dataset: $\Rightarrow$ CIFAR 10

of training examples: 5000

of test examples = 10,000

of Transformer blocks in a stream = 4

of Attention heads = 4

CIFAR 10

Color images

of

size $\Rightarrow \underline{\underline{32 \times 32}}$

number of classes = 10

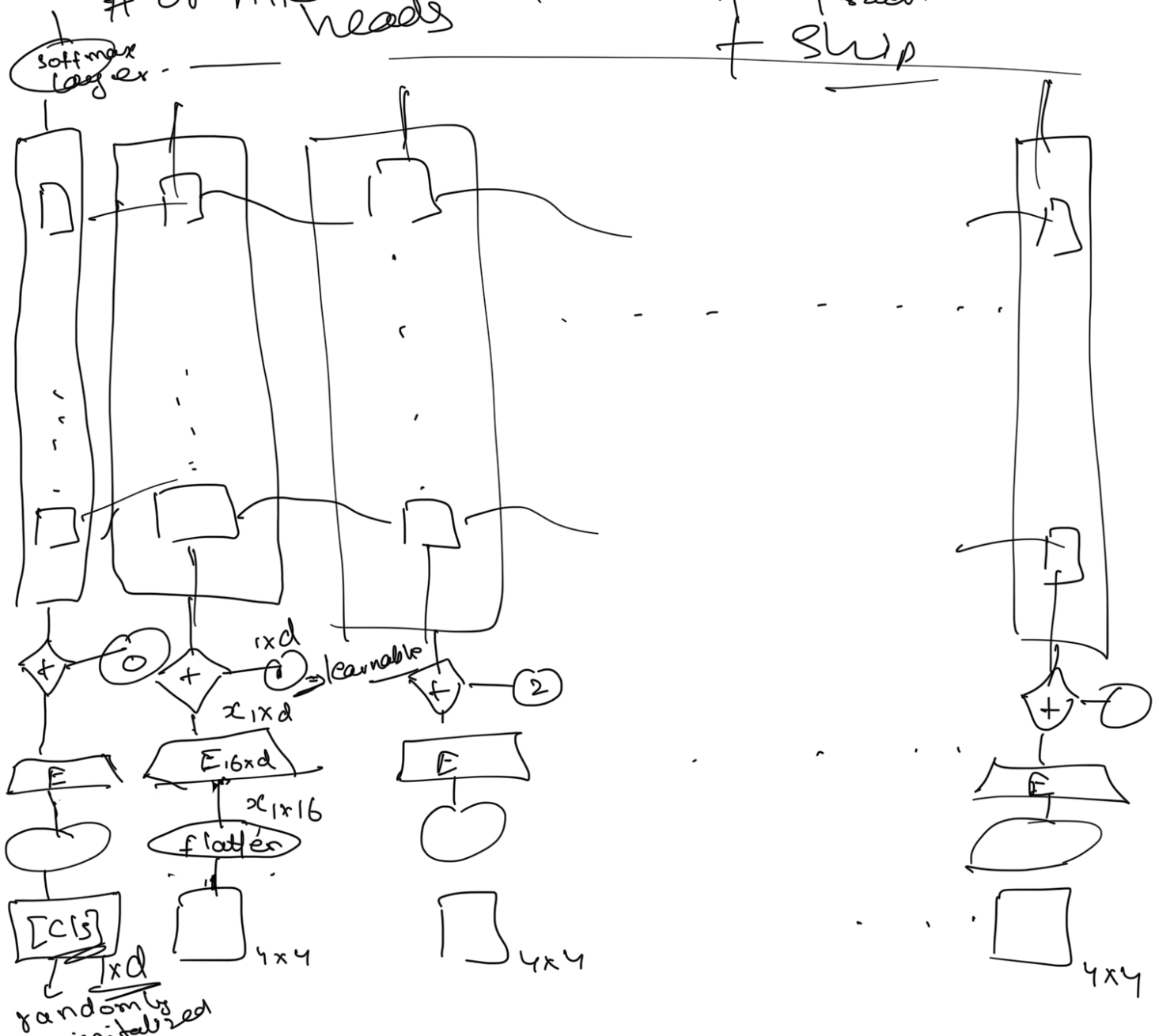
— Aeroplane

+ dog

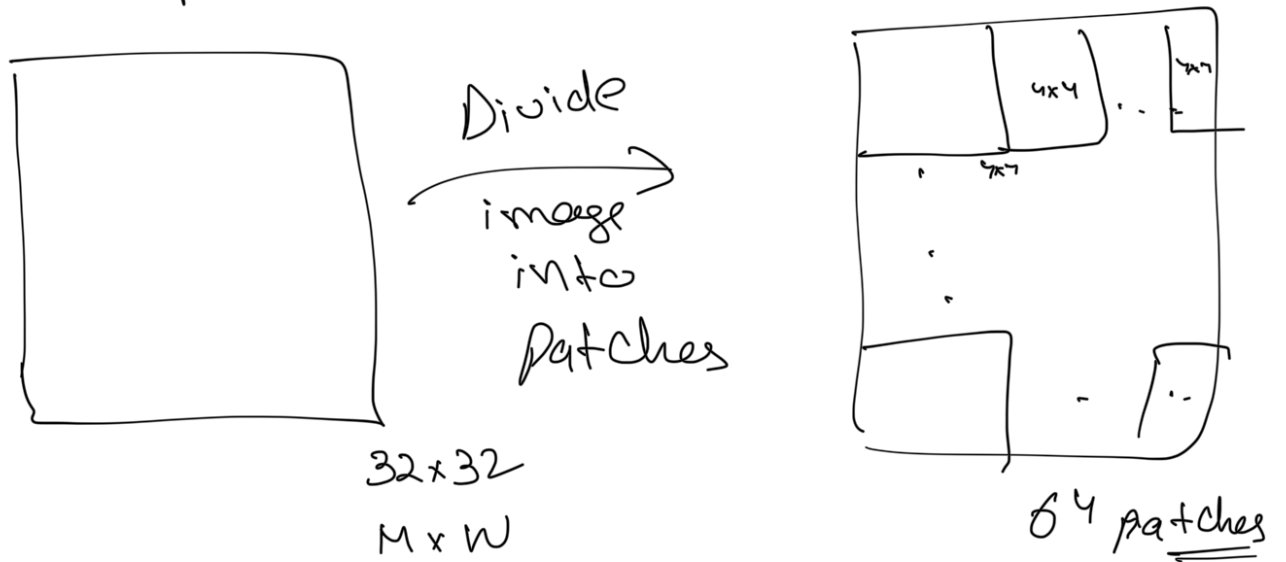
+ Cars

+ Touch

f slip



ViT Input



$$\text{patch size} = P$$
$$\text{patch dim} = \left(\frac{M}{P} \times \frac{W}{P} \right)$$

$$\# \text{ of patches} = \text{Context window length}$$

For classification using transformers

↓
Add a new stream at the beginning.

↓
The input token of the stream is a $[CLS]$

token =

↓
of Parameters is Very large

Eg: $\Rightarrow$ # of Attention heads = 4

of transformer

sub block in

each stream = 4

of training images = 50,000

of Parameters 550k

↓
Clearly would lead to
over fitting
in many cases

↓
way to resolve: Regularization

① $\Rightarrow$ Data Augmentation:

↓
Adding samples to original
training set.

...

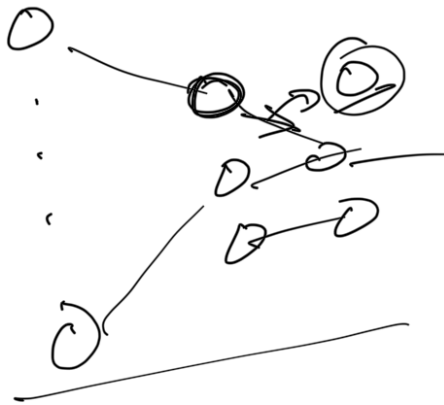
1) Rotate an image and add to training set.

2) Horizontal flip

(2)

Dropout : $\Rightarrow 0.1$

The network randomly drops a ratio of neuron outputs.



means $\searrow$
10% of neurons will be randomly switched off during training.

$\Rightarrow$ My model is still overfitting at a dropout of 0.2!

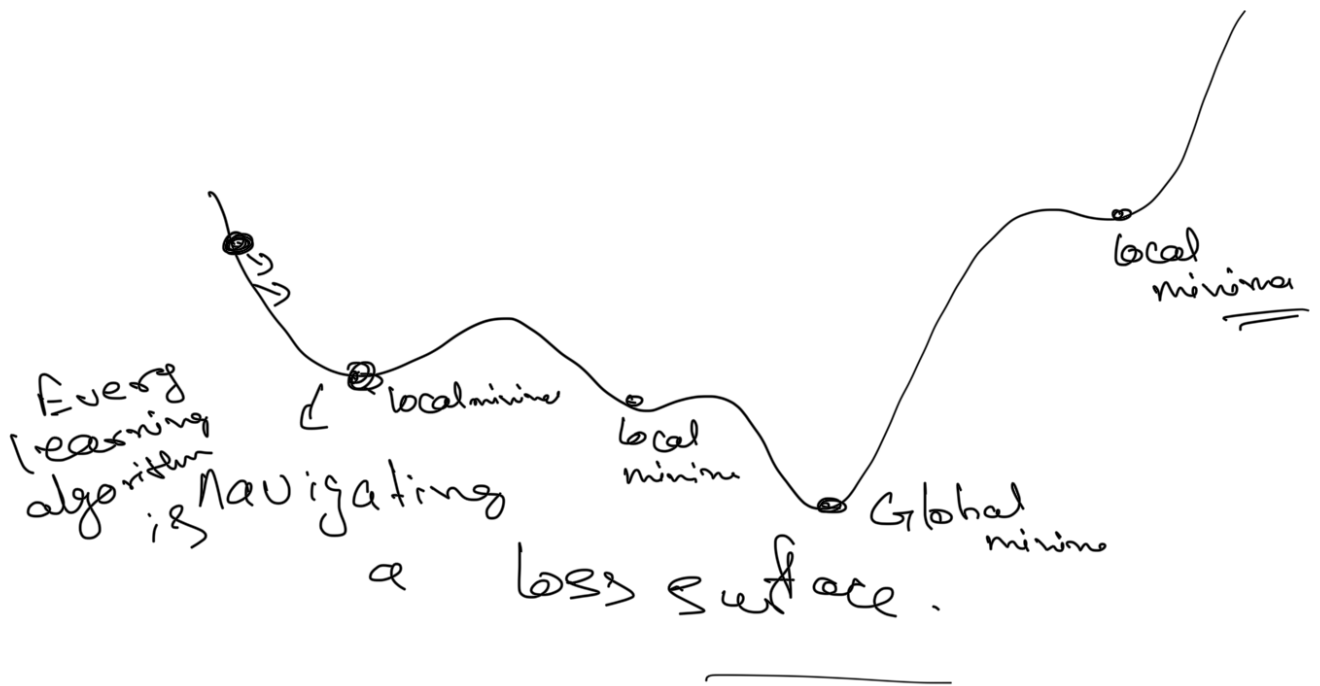
Should I increase or decrease dropout?

Give reason,

Learning rate Scheduler :

Till now we have kept
the learning rate same
throughout training.

Is this a good strategy?



Scheduler

↓
Varies learning rate
as training progresses

- High learning rate in beginning
- low learning rate in end.

(beginning, end) $\rightarrow$ use loss
 $\rightarrow$ # of epochs

Why was this scheduler not
so much important for CNNs?

(
of parameters has
increased drastically
 $\uparrow$
same learning algorithm
is navigating
a larger dimensional
hypersurface